Naive Bayes Classifier Project Report

Garrett Nix

October 15, 2026

1 Introduction

In this assignment, concepts learned from class, specifically the implementation of a Naive Bayes model, is investigated. The task of observing the performance of a Naive Bayes classifier as the training size increases is performed on bank data obtained from the UCI Machine Learning Repository. A table measuring this performance is discussed.

2 Methodology

The data obtained for this project is based on bank data obtained from the University of California Irvine Machine Learning Repository. This data consists largely of categorical and binary features, having only 6 out of the 16 features being numerical. Loading the data into a dataframe in Python using the Pandas library, a delimeter set to ';' allowed for simple manipulation and functionality for data preprocessing. This preprocessing consists of one-hot encoding the categorical and binary columns into numerical values using the `pd.get_dummies()` method with the encoding data type set to integer.

3 Results

Once this preprocessing was completed, the assignment was followed for each of the training/test splits specified fed into a NB classifier. Gathering the various averages and standard deviations for both the accuracy and f1 score provides the results seen in both Figure 1 and Table 1. It should be noted that each iteration signifies $10x\%$ training size, where x is the iteration.

Table 1: Average and standard deviations for accuracy and f1 score as training size increases.

Trial	Average accuracies	Accuracy STD	Average f1 scores	f1 STD
1	0.8575	0.0072	0.8510	0.0093
2	0.8600	0.0035	0.8544	0.0047
3	0.8590	0.0068	0.8529	0.0084
4	0.8616	0.0037	0.8563	0.0045
5	0.8624	0.0031	0.8572	0.0037
6	0.8619	0.0027	0.8566	0.0031
7	0.8619	0.0021	0.8567	0.0027
8	0.8643	0.0032	0.8594	0.0035
9	0.8617	0.0047	0.8562	0.0053

It can be seen that an increase in training size does provide slight improvments in average accuracies and f1 scores, however the standard deviations vary per trial run. Using Table

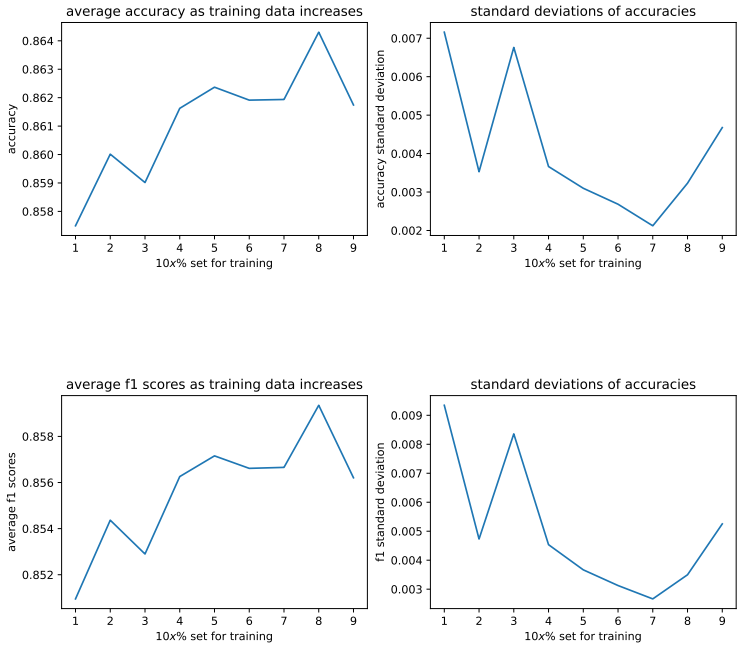


Figure 1: Visual of averages and standard deviations as training size increases.

1 suggests that a training size of 50% and 60% provides high averages for both accuracy and F1, with low standard deviations. This is indicative of a well-performing model that provides the best fit overall, and should be used for further classifications.

3.1 Improving model performance

Reanalyzing the data, there exists values within some of the columns being labeled as "unknown". Removing these values from the dataset and rerunning the model we obtain the results seen in Table 2 and Figure 2.

This suggests that removing these unknown values provides a general increase in accuracy and F1 score, while dropping the standard deviation in a similar manner. A training set of 50% or 70% seem to provide the best fits overall for this new data.

4 Conclusion

Overall, this project aimed to provide a solidification of concepts learned so far in the course. Using data that provides a model with only slight improvements aids in ensuring critical thinking is used when working with these models. This ensures that when applying these concepts in real-world problems that we're not just "pressing the button", but rather understanding what it is we're trying to answer, and what is the best solution we can come up with to provide the best predictions.

Table 2: Average and standard deviations for accuracy, F1 with "unknown" values removed.

Trial	Average accuracies	Accuracy STD	Average f1 scores	f1 STD
1	0.8647	0.0053	0.8607	0.0065
2	0.8669	0.0028	0.8638	0.0036
3	0.8674	0.0034	0.8642	0.0044
4	0.8667	0.0047	0.8635	0.0059
5	0.8683	0.0023	0.8654	0.0027
6	0.8675	0.0024	0.8643	0.0031
7	0.8672	0.0021	0.8645	0.0028
8	0.8692	0.0024	0.8666	0.0028
9	0.8690	0.0050	0.8661	0.0053

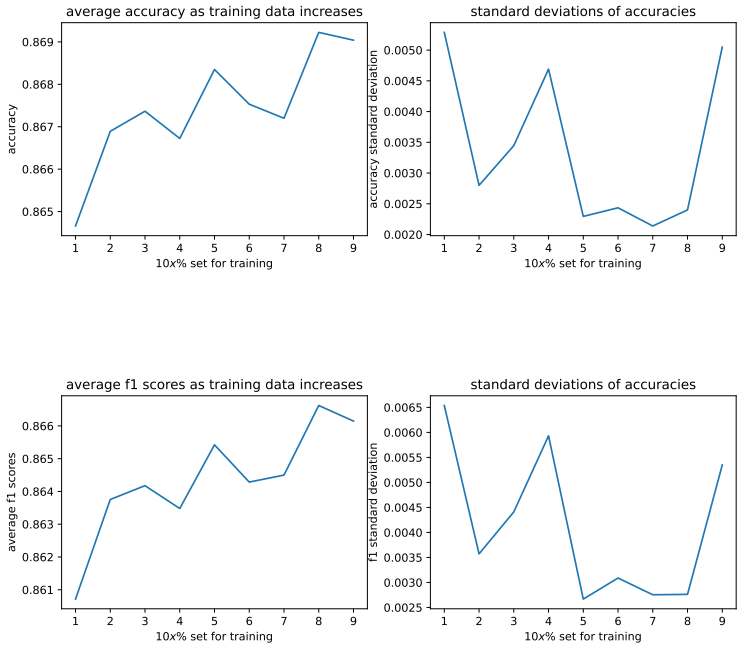


Figure 2: Accuracies and standard deviations with "unknown" values removed.